BIG DATA · UDEM

Big Data e Ingeniería de Datos

Clase 2 · Cómo se lee big data y SQL

Posgrado · UdeM · 4 nov 2026

239.5 M

viajes de alquiler con conductor de 2024, en crudo (sin procesar). Clase 1: gold, el resumen, 15 909 filas. Para medir el motor: una tabla estándar más grande.

NYC TLC

Sumar 600 M de líneas tarda 0.6 s en caliente y 2.5 s en frío

```
SELECT SUM(L_EXTENDEDPRICE) AS IMPORTE
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF100.LINEITEM;
```

Caliente: datos en caché del warehouse (cómputo de Snowflake); frío: sin ellos. TableScan (paso que lee la tabla) leyó 1.83 GB. ¿Cómo cabe la columna y se suma tan rápido?

LINEITEM es una tabla de TPC-H, banco estándar que simula un mayorista

TPC-H simula un mayorista. Los labs usan SF1; las demos de escala, SF100 y SF1000.



Filas de LINEITEM por factor de escala (SF)

SF1 · 6 M	SF10 · 60 M	**SF100 · 600 M**	SF1000 · 6 000 M

PART y SUPPLIER no se usan. 1:N: una fila de la izquierda tiene muchas filas a la derecha. SF1 pesa 165 MB en LINEITEM.

SF: factor de escala. LINEITEM: 6 M de filas en SF1; 600 M en SF100; 6 000 M en SF1000.

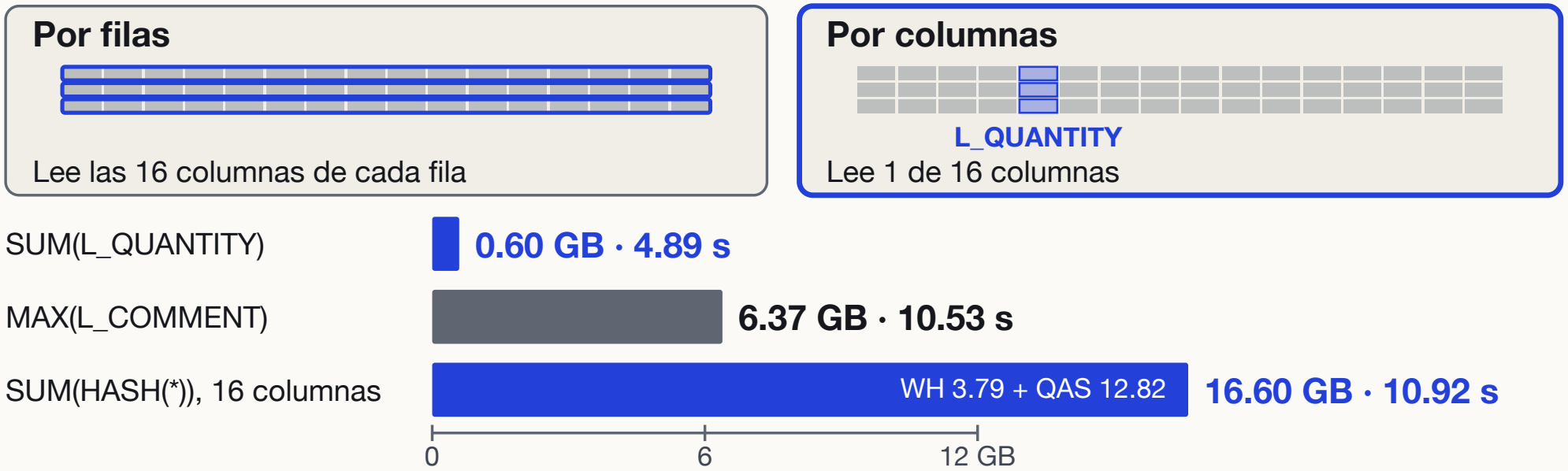
Leer rápido exige resolver ocho problemas: síntoma con dato, idea de ingeniería y respuesta de Snowflake

Problema	Síntoma con dato	Idea de ingeniería	Snowflake
1 **Columnar**	0.60 GB frente a 6.37 GB	Leer solo columnas	**Micro-particiones**
2 **Pruning**	134 de 10 336 particiones	Omitir por rango	**Rango por columna**
3 **Escalar**	10× filas, unas 9× tiempo	Separar cómputo y datos	**Warehouses por talla**
4 **Aislar cargas**	Carga de 6.0 s con consulta de 0.79 s	Cómputo por carga	**Un warehouse por carga**
5 **Costo y elasticidad**	Medium: 1.45× más rápido, 2.8× el costo	Pagar lo encendido	**Auto-suspend y caché**
6 **Datos crudos**	15 de 24 columnas usadas	Esquema flexible	VARIANT
7 **Transformación**	Filtros, 2 uniones, aplanado	Código con pruebas	SQL; pipeline aparte
8 **Formatos abiertos**	5.4 GB de Parquet en S3	Archivos abiertos	Tablas Iceberg

Warehouse: cómputo de Snowflake. X-Small: talla menor, 1 crédito (unidad de cobro) por hora.

Solo se leen las columnas pedidas: 0.6 GB numérica, 6.4 GB texto, 16.6 GB todas

Las columnas se guardan por separado: la consulta lee solo las que nombra.

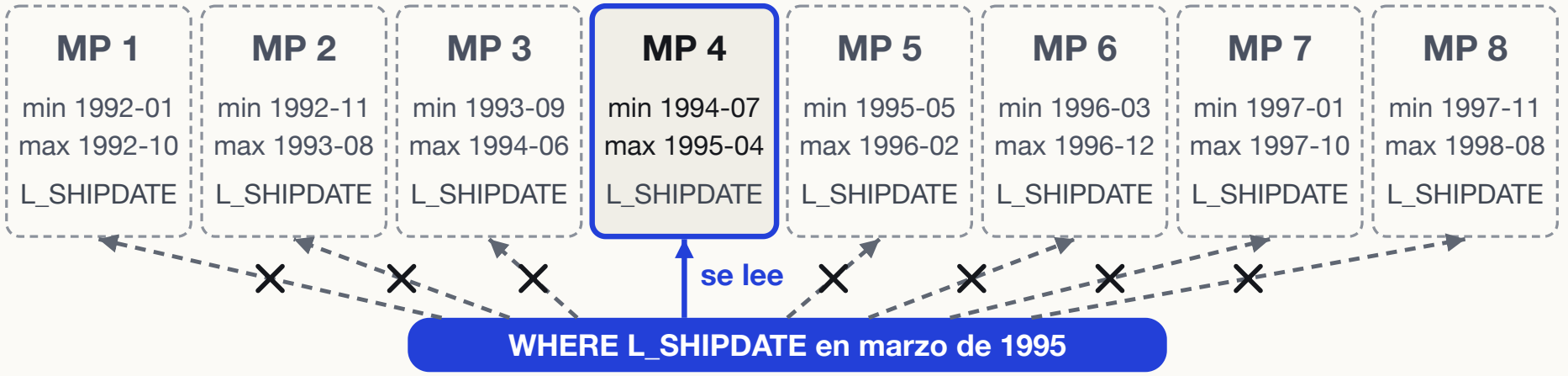


TPCH_SF100, X-Small, caché fría. WH: warehouse; QAS: escaneo fuera del warehouse.

Problema 1 de 8: columnas. QAS: servicio que hace parte del escaneo fuera del warehouse (WH).

Para saltar filas, cada micro-partición guarda su rango; un mes lee 0.25 GB, no 19.2 GB

El orden de carga agrupa las fechas: cada micro-partición guarda el rango de cada columna y el filtro salta las que no encajan.



134 de 10 336 micro-particiones · **19.2 GB → 0.25 GB** · **6.0 s → 2.0 s**

SF1000, X-Small, un mes de L_SHIPDATE. Sin filtro: 6.0 s, media de las ejecuciones 3 a 5; 19.2 GB

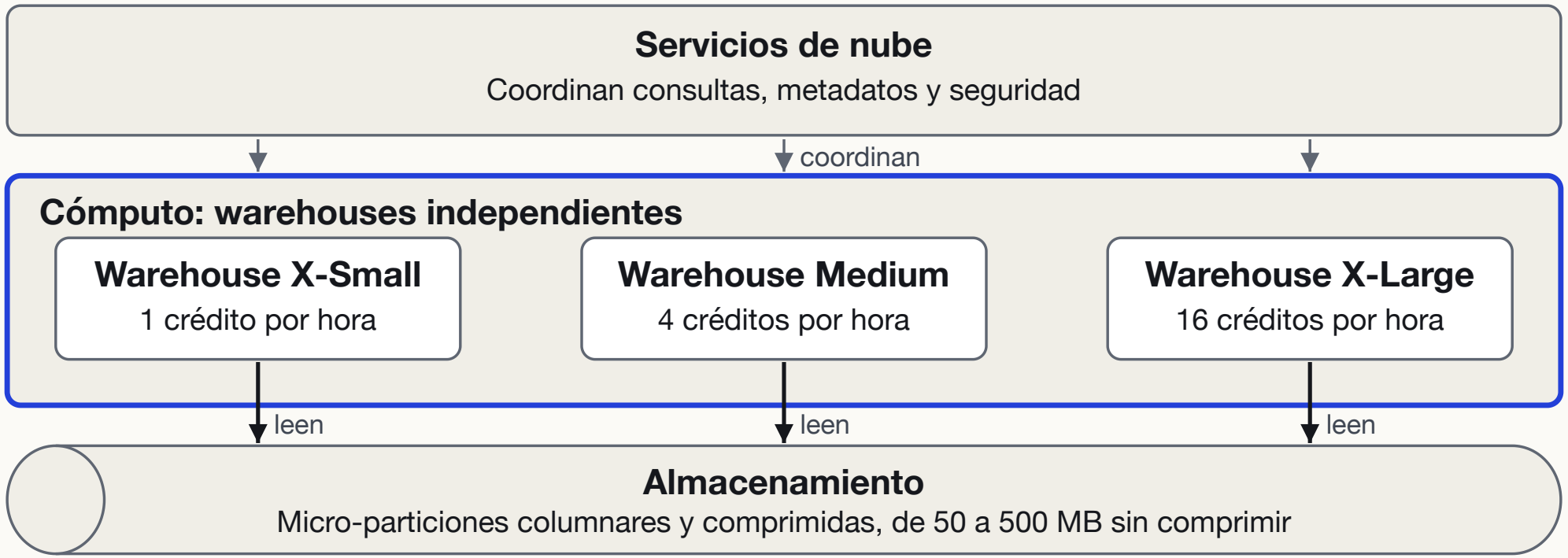
Límite: solo poda si las filas llegan ordenadas por esa columna. En L_SHIPDATE sí; en L_QUANTITY cada micro-partición abarca casi todos los valores. Rangos ilustrativos.

Problema 2 de 8. Micro-partición: bloque de 50–500 MB sin comprimir. Pruning (poda): omitir las descartadas.

Con el mismo warehouse, 10 veces más filas son unas 9 veces más espera: hay que sumar cómputo

Escala	Filas	Tiempo medio en X-Small	GB totales
SF100	600 M	0.65 s	1.83 GB
SF1000	6 000 M	6.0 s	19.2 GB

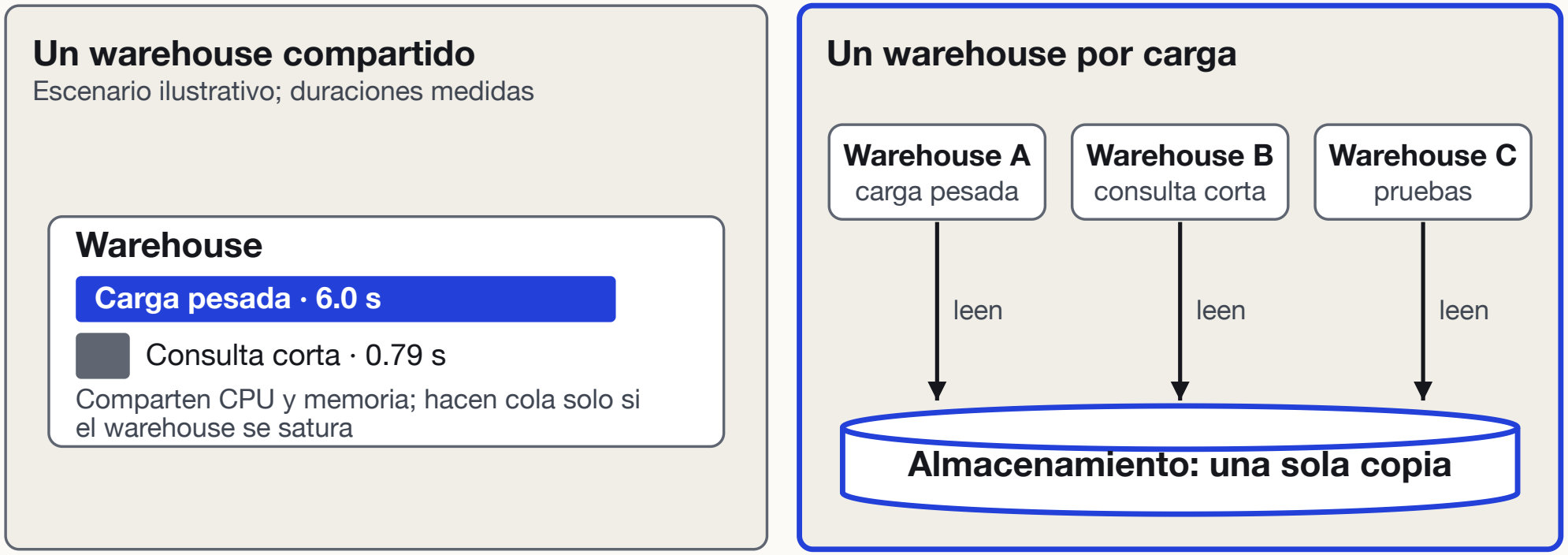
Para sumar cómputo sin tocar los datos, Snowflake separa almacenamiento, cómputo y servicios de nube



Cada capa escala por su cuenta: más datos, solo almacenamiento; más consultas, más warehouses; más potencia, una talla mayor.

Problema 3 de 8. De X-Small (1 crédito por hora) a 6X-Large (512 créditos por hora).

Con cómputo separado, varios warehouses leen los mismos datos y una carga pesada no frena a las demás

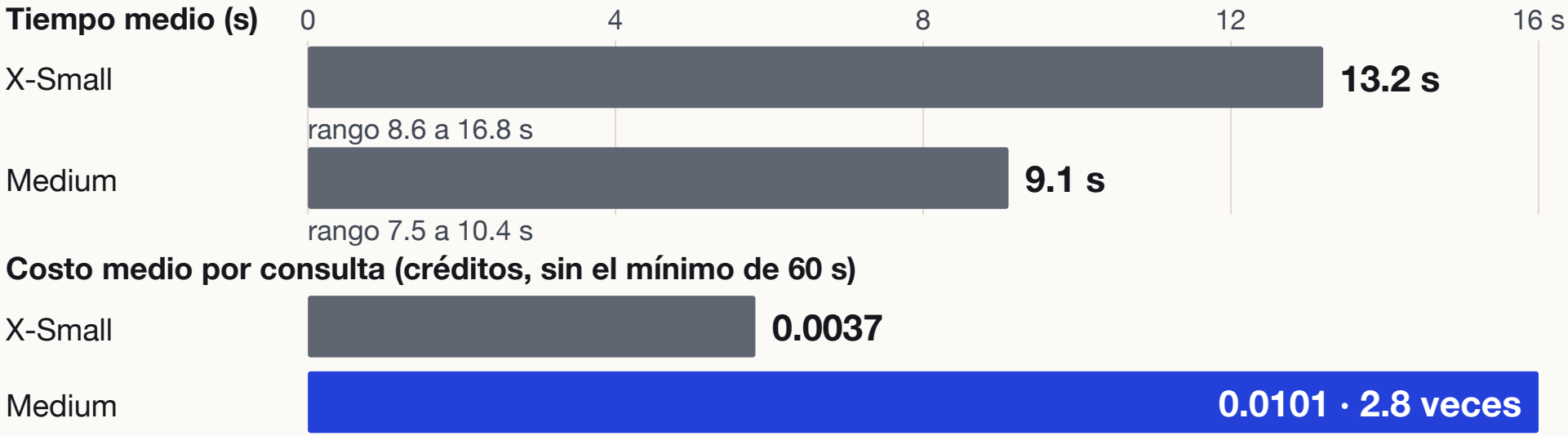


Cada warehouse tiene su propio tiempo y no afecta a los demás; cada uno encendido factura sus créditos.

Problema 4 de 8: aislamiento. Cada warehouse encendido factura sus propios créditos.

Medium fue 1.45 veces más rápido que X-Small y costó 2.8 veces más por consulta

Misma consulta y mismo volumen: 9 144 de 10 336 micro-particiones en las ejecuciones completas; tres ejecuciones por talla.



Medium fue 1.45 veces más rápido y costó 2.8 veces más. Con n = 3, la diferencia de tiempo es indicativa.

Problema 5 de 8. Caché de resultados, consulta repetida: 2.14 s, luego 0.05 s y 0 bytes.

Snowflake resuelve cinco problemas; los otros tres son decisiones del curso

Problema	Snowflake	Dónde sigue
Crudos	VARIANT	Clase 4: lake en S3
Transformación	Puede, con SQL o Snowpark	Clases 5-6: Spark y Airflow
Formatos	Tablas Iceberg	Clase 4: Parquet en S3

Los otros motores también omiten datos; cambia el cobro: tiempo encendido o bytes leídos

Motor	Omite datos	Cobro
Snowflake	Rango por micro-partición	Tiempo
Redshift (AWS)	Mín/máx por bloque	Tiempo
BigQuery (Google)	Particiones, clustering	Bytes (on-demand)
Databricks SQL	Mín/máx por archivo	Tiempo

Las clases 4 a 6 corren en AWS: verifica tu cupo

AWS

1. Bootstrap: crea los recursos de AWS del curso.
2. Necesita cuatro cuotas (paso 2.2 de la guía).
3. Si falta cupo: Request increase (paso 2.3).
4. Rojo: Lab 1. Verde claro: bootstrap hecho.

Lab 1: tus primeras consultas sobre TPC-H

Snowsight

1. Copy raw file de lab1.sql; corre el Paso 0 con ▶, no con Run All.
2. EJ1 a EJ6: reemplaza SELECT NULL.
3. EJ6: abre el Query Profile (plan ejecutado).
4. Otra hoja: lab1_verificacion.sql, Run All.

DESCANSO

Hasta aquí, cada consulta devolvió filas o un solo número. ¿Cómo se resume por grupo y en qué orden se evalúa una consulta?

GROUP BY resume por grupo: WHERE filtra filas antes y HAVING filtra grupos después

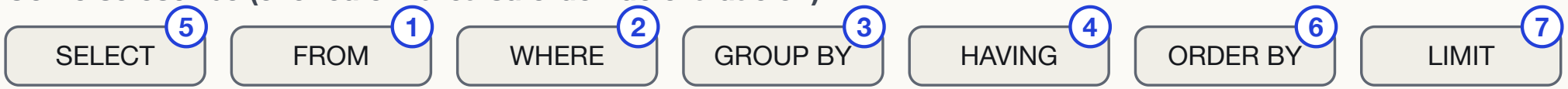
```sql
SELECT O_CUSTKEY, COUNT(*) AS PEDIDOS
FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE O_ORDERDATE >= '1995-01-01' AND O_ORDERDATE < '1996-01-01'
GROUP BY O_CUSTKEY
HAVING COUNT(*) >= 8
ORDER BY PEDIDOS DESC, O_CUSTKEY
LIMIT 5;
```

GROUP BY agrupa por cliente; COUNT(*) cuenta pedidos por grupo. WHERE (activa la poda, slide 7) deja 228 637 pedidos de 1995; HAVING, los 622 clientes con 8 o más.

WHERE va antes de HAVING: el motor evalúa FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY y LIMIT

Se escribe empezando por SELECT; se evalúa empezando por FROM.

Cómo se escribe (el círculo marca su orden de evaluación)



Cómo se evalúa



Filas que quedan (SF1, consulta de la slide 17)

1	2	3	4	5	6	7
1 500 000 pedidos	**228 637** de 1995	**86 573** grupos	**622** con 8 o más	**622** filas, 2 columnas	**622** ordenadas	**5** filas

WHERE no ve agregaciones: aún no existen. ORDER BY sí ve los alias del SELECT.
Es un orden lógico, no físico.

Orden lógico, no físico: el optimizador puede reordenar si el resultado no cambia.

JOIN va dentro de FROM: unir pedidos con líneas repite cada pedido y multiplica la suma por 5

Un pedido (uno) tiene varias líneas (muchos): al unir, el pedido se repite por cada línea.

ORDERS
pedido A · O_TOTALPRICE 100

LINEITEM
A-1
A-2
A-3

JOIN →

Resultado del JOIN
A-1 · **100**
A-2 · **100**
A-3 · **100**

SUM(O_TOTALPRICE) = 300; el pedido vale 100

1 500 000 → 6 001 215

filas: pedidos contra filas tras el JOIN (× 4.0)

226.8 → 1 134.4 mil millones

SUM(O_TOTALPRICE) sin JOIN contra con JOIN (× 5.0)

El monto crece más que las filas porque los pedidos grandes tienen más líneas y se repiten más.

INNER: solo parejas; LEFT: todas las de la izquierda (50 004 clientes sin pedidos).

La cardinalidad, filas de una por fila de la otra, decide si el JOIN multiplica

Unión	Relación	Filas
ORDERS–CUSTOMER	Muchos a uno	Por pedido
ORDERS–LINEITEM	Uno a muchos	1.5 M → 6.0 M
LINEITEM–PARTSUPP	Muchos a muchos	60.0 M → 239.9 M

Lab 2: la cardinalidad, meta 6 de 6

Snowsight

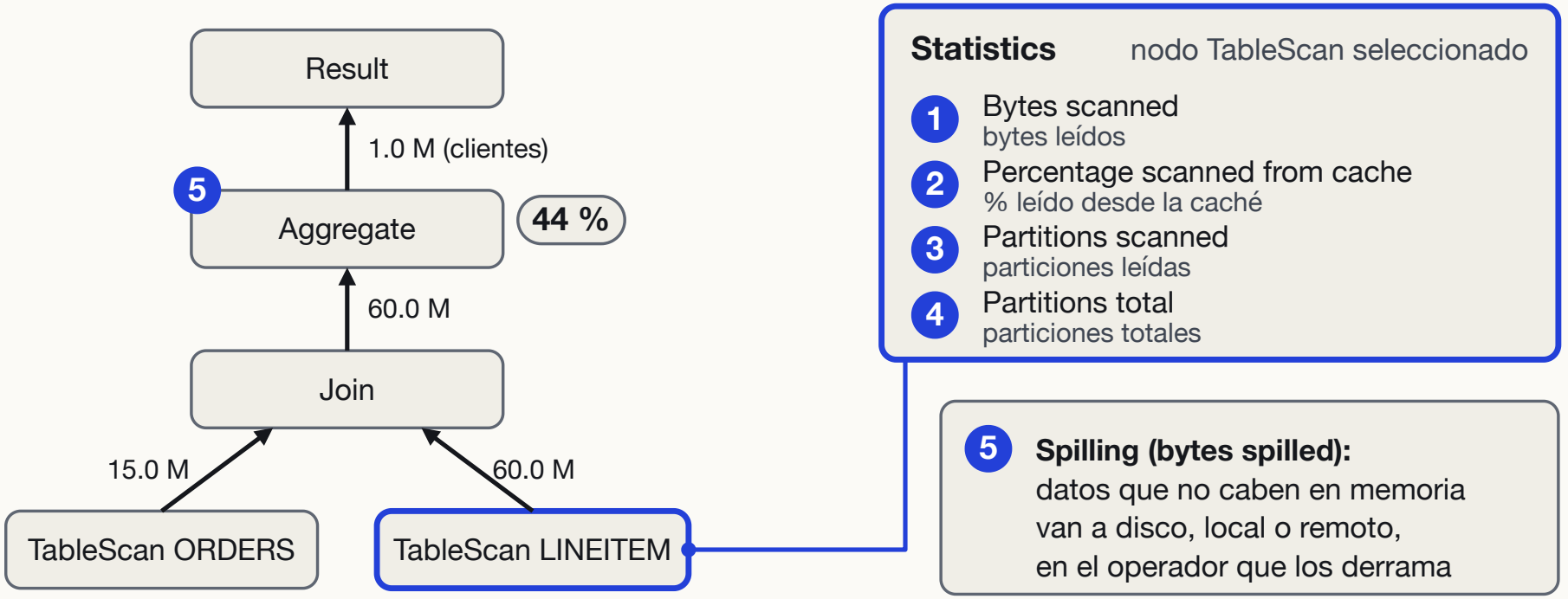
1. Copia lab2.sql; Paso 0.
2. EJ1 a EJ6 (★, básico).
3. EJ7 y EJ8 (★★): duplicación y corrección.
4. COUNT(DISTINCT) cuenta cada clave una vez: AUTOMOBILE 297 453 filas, 19 876 clientes.
5. lab2_verificacion.sql, Run All.

Tras preguntar, se mide: el Query Profile es el plan ejecutado

Snowsight

1. Apaga la caché de resultados (reutiliza consultas idénticas): `ALTER SESSION SET USE_CACHED_RESULT = FALSE;`
2. Ejecuta; Query Details > tres puntos > View Query Profile.
3. Otra hoja: Monitoring > Query History.

Cada operador es un paso del plan; el árbol muestra su tiempo y sus filas



> Se lee de abajo hacia arriba: filas que suben, tiempo por nodo y, en el nodo elegido, lectura y derrame.

Árbol de unir y agregar (SF10): Aggregate, 44 % del tiempo. Aggregate agrupa; Sort ordena.

Leído el árbol, cada síntoma apunta a una causa y a una decisión distinta

Síntoma	Causa	Decisión
Particiones leídas = totales	Filtro sin poda	**Filtrar por columna ordenada**
Muchos bytes en TableScan	Columnas de más	**Pedir solo las columnas que necesitas**
Join entrega más filas que su entrada mayor	Llave incompleta	**Revisar qué representa cada fila y la clave de unión**
Join o Aggregate dominan	Muchas filas antes de unir	**Agregar antes de unir**
Bytes spilled	Falta memoria	**Menos datos o más talla**
Sort domina sin necesidad	ORDER BY intermedio	**Quitarlo**

Pedir 3 columnas: 0.45 GB frente a 1.72 GB y 5 veces menos tiempo; mismas 116 particiones.

El pruning depende del orden de filas: en SF100, YEAR() podó igual que el rango

Filtro	Escala	Particiones
YEAR(L_SHIPDATE) o rango 1995	SF100	171 de 1 022
L_QUANTITY = 10	SF100	1 022 de 1 022
Rango 1995	SF10	116 de 116

Unir por llave incompleta multiplica las filas por 4 sin error: el Join lo delata

LINEITEM con PARTSUPP (4 proveedores por pieza), SF10	Solo PARTKEY	PARTKEY y SUPPKEY
Filas	239.9 M	60.0 M
Join: % del tiempo total	89 %	63 %

Si hay spilling, Medium lo hizo 4.4 veces más rápido, casi al mismo costo

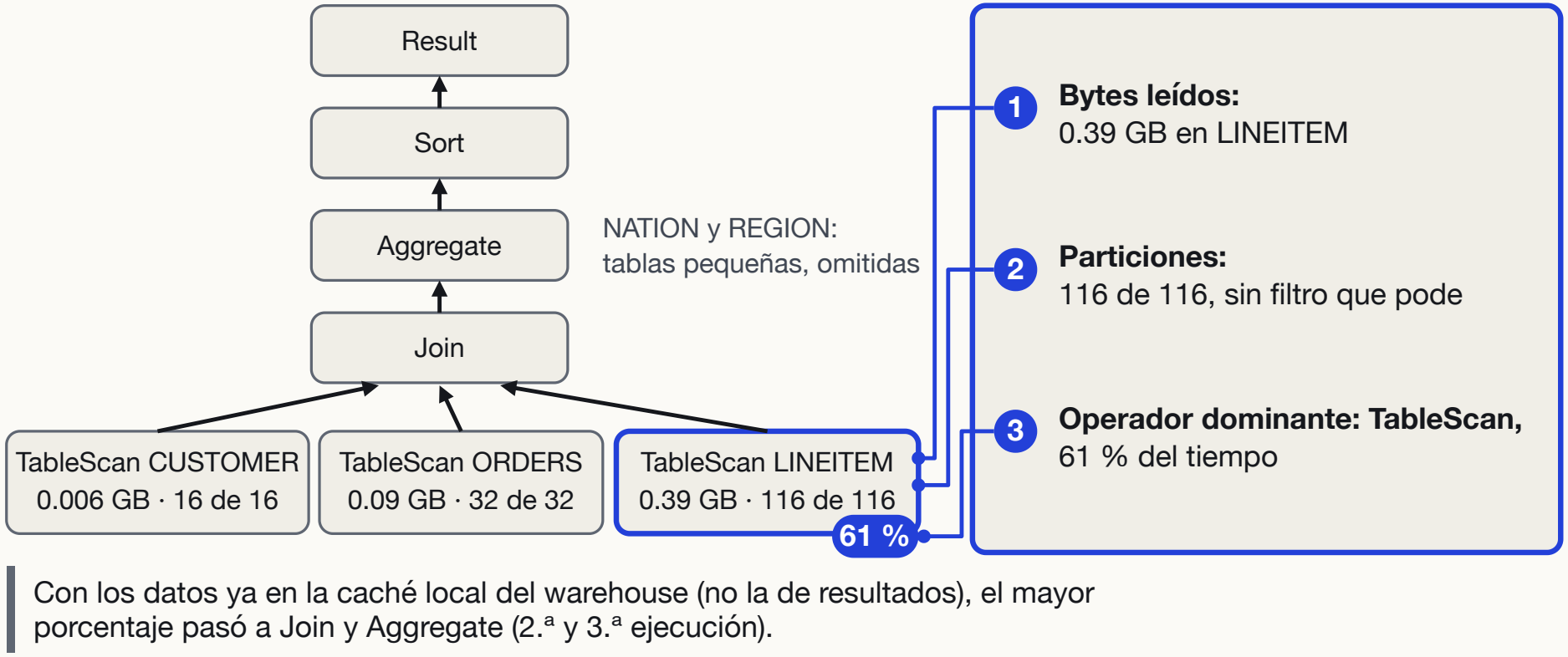
Ventana sobre 600 M filas	X-Small	Medium
Bytes spilled local (a disco)	4.73 GB	0
Tiempo	48.4 s	11.0 s
Créditos (sin mínimo de 60 s)	0.0134	0.0122

Mini-caso de una dirección comercial: hoy, Q1 y Q4 sobre SF10

Snowsight

1. caso.sql: Paso 0 con ▶.
2. Q1: ventas por región, 1995.
3. Q4: importe por envío y región.
4. caso_verificacion.sql: Run All.
5. Q4 en OK: abre su Query Profile.

Sin filtro, Q4 lee 116 de 116 micro-particiones: LINEITEM entera, 0.39 GB



> Con los datos ya en la caché local del warehouse (no la de resultados), el mayor porcentaje pasó a Join y Aggregate (2.ª y 3.ª ejecución).

Q4, SF10, X-Small, primera ejecución, en frío: domina TableScan. ¿Qué cambiarías para que lea menos?

Leer rápido es leer menos: columnas, particiones, caché

Problema	Síntoma con dato	Idea de ingeniería	Dónde sigue
1 **Columnar**	0.60 GB frente a 6.37 GB	Leer solo columnas	**Visto hoy: Lab 1, Lab 2 y mini-caso**
2 **Pruning**	134 de 10 336 particiones	Omitir por rango	**Visto hoy: Lab 1, Lab 2 y mini-caso**
3 **Escalar**	10× filas, unas 9× tiempo	Separar cómputo y datos	**Visto hoy: demos**
4 **Aislar cargas**	Carga de 6.0 s con consulta de 0.79 s	Cómputo por carga	**Visto hoy: demos**
5 **Costo y elasticidad**	Medium: 1.45× más rápido, 2.8× el costo	Pagar lo encendido	**Visto hoy: demos**
6 **Datos crudos**	15 de 24 columnas usadas	Esquema flexible	Clase 4: lake en S3
7 **Transformación**	Filtros, 2 uniones, aplanado	Código con pruebas	Clases 5 y 6: Spark y Airflow
8 **Formatos abiertos**	5.4 GB de Parquet en S3	Archivos abiertos	Clase 4: Parquet en S3

600 M de valores en 0.6 s: una columna comprimida (1.83 GB), 1 022 micro-particiones en paralelo. Clases 4 a 6: 239.5 M viajes.